Design Class Diagram - PacMan in the Multiverse (Extended System)

propertiesLoader
+ loadPropertiesFile(fileName: String): Properties

ConfigurationLoader
+ load(properties: Properties): GameConfiguration
- parseLayout(layout)
- validateDimensions(properties, width, height)
- validateRows(rows, width)
- validateLocations(...)
- validateLocation(...)
- readScriptedMoves(...)

creates

loads

Game
- nbHorzCells : int
- nbVertCells : int
- gameController: GameController
- gameCallBack: GameCallback

+ Game(gameCallback: GameCallback, properties: Properties)
+ act() : void
- updateTitle(): void
+ runApp(): String
- setupActorLocation(): void
- drawGrid() :void
- putPill(): void

Driver
+ DEFAULT_PROPERTIES_PATH$: String

+ main(String[] args): void

GameCallback
- logBuilder: StringBuilder

+ pacManLocationChanged(...): void
+ monsterLocationChanged(...): void
+ pacManEatItems(...): void
+ getAllLog(): String
+ endOfGame(result): void
+ effectTriggered(...) : void

<<JGameGrid>>
GameGrid

PacmanGameGrid
- mazeArray: int[][]

+ getCell(location: Location): int
- toInt(char) : int

PacActor
- score: int
- nbPills: int
- gameController: GameController

+ eatPill(location: Location): void

+ addScore(value: int): void

+ addPill(): void

PacmanController
- pacActor: PacActor

+ keyRepeated(keyCode: int): void

drives via keys

getScore()

GameConfiguration
- width: int
- height: int
- mazeLayout: List<String>
- pacManLocation: Location
- pacManDirection: Location.CompassDirection
- monsterConfigurations: List<MonsterConfiguration>
- itemConfigurations: List<ItemConfiguration>

GameController
- activeItems: List<Item>
- itemActorLocations: List <ActorLocation>
- monsters : List<Monster>
- effectManager : Effectmanager
- usesConfigurableMap : boolean
grid: PacManGameGrid
pacActor: PacActor

+ incrementTick() : void
+ stopMonsters(): void
+ checkEndGame(): void
- countPillsAndItems(): int
+ setupPillAndItemsLocations(): void
+ processItem(location: Location): void

ActorLocation
- Actor actor
- location: Location
- direction: Location.CompassDirection

EffectManager
- gameCallback: GameCallback
- monsters: List<Monster>

+ triggerFury(monsters: List<Monster>): void

+ triggerFreeze(monsters: List<Monster>): void

+ isFrozen(monsters: List<Monster>): boolean

<<Enumeration>>
MonsterType
Troll
Orion
Wizard

+ getImageName(): String

MonsterConfiguration
- monsterType: MonsterType
- scriptedMonsters : List<String>
- id: int
- location: Location
- direction: Location.CompassDirection

ItemConfiguration
+ type: ItemType
+ location: Location

MonsterFactory

+ createMonsters(gameController, config): List<ActorLocation>

ItemFactory

+ createItems(type : ItemType, location : Location) : Item

<<interface>>
Item

+ consume(pacMan: PacActor, gc: GameController): void

<<Enumeration>>
ItemType
ICECUBE
PILL
GOLDPIECE

<<abstract>>
Monster
+ currentEffect: MonsterEffect
+ movementStrategy : MovementStrategy
+ gameController: GameController

+ act(): void
+ *getType(): MonsterType*
+ computeBaseNextLocation() : Location
+ applyEffect(MonsterEffect) : void

<<interface>>
MonsterEffect
+ modifyMovement(...): Location
+ tick(): void
+ isExpired(): boolean

FuriousEffect
- ticksRemaining: int

+ modifyMovement(baseStrategy, monster, grid): Location <<override>>
+ tick(): void <<override>>
+ isExpired(): boolean <<override>>

IceCube
- SCORE : int

- consume(paceman, gc): void

GoldPiece
- SCORE : int

- consume(paceman, gc): void

Pill
- SCORE : int
- location : Location

- consume(paceman, gc): void

Orion

+ getType(): MonsterType <<override>>

Troll

+ getType(): MonsterType <<override>>

Wizard

+ getType(): MonsterType <<override>>

<<abstract>>
IceEffect
- tickRemaining: int

+ tick(): void <<override>>
+ isExpired(): boolean <<override>>

FrozenEffect

+ modifyMovement(baseStrategy, monster, grid): Location <<override>>

+ tick(): void <<override>>

WizardIceEffect
- moveInThisTick: boolean

+ modifyMovement(baseStrategy, monster, grid): Location <<override>>

<<interface>>
MovementStrategy

+ determineMovement(monster : Monster, grid: PacManGameGrid): Location

MoveDecision
+ direction: Location.CompassDirection
+ numberOfCells: int
+ requiresTurn: boolean

+ getDirection(): Location.CompassDirection
+ getNumberOfCells(): int
+ TurnRequires: boolean

TrollMovementStrategy
- randomiser : Random

+ determineMovement(monster : Monster, grid: PacManGameGrid): Location <<override>>

OrionMovementStrategy
- patrolRoute: List<Location>

- routeIndex: int

+ determineMovement(monster : Monster, grid: PacManGameGrid): Location <<override>>
- getNextStepBFS() : Location

WizardMovementStrategy
- randomiser : Random

+ determineMovement(monster : Monster, grid: PacManGameGrid): Location <<override>>
+ tryWallPhase(...) : Location
+ createFreezeEffect() : MonsterEffect